Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Automatic Control Laboratory

Masterthesis

Non-Convex Projection Layers via Sequential Quadratic Programming

Nicholas Behr
September 2025

Advisors
Panagiotis D. Grontas, Antonio Terpin, Efe C. Balta, John Lygeros

Abstract

Traditional neural networks rely on soft constraints that penalize but do not guarantee constraint satisfaction, posing unacceptable risks in safety-critical applications. This thesis presents a novel nonconvex projection layer approach, leveraging sequential quadratic programming (SQP), which enforces nonconvex hard constraints directly within neural networks, thereby ensuring mathematical guarantees of constraint satisfaction.

The core of this work is a differentiable optimization layer that projects a network's output onto a feasible set defined by nonlinear and potentially nonconvex constraints. The forward pass of this layer solves a parametric optimization problem using Sequential Quadratic Programming (SQP), with an efficient operator-splitting solver (OSQP) for its quadratic subproblems. For the backward pass, we use the Implicit Function Theorem (IFT) to compute exact gradients of the solution with respect to the layer's inputs. This method avoids the high memory cost and numerical instability associated with unrolling the solver's iterations for gradient computations, enabling stable end-to-end training.

We demonstrate the merit of the SQP Layer in the context of rollout projection for control systems. A neural network proposes a trajectory, which the SQP Layer then projects to the closest feasible trajectory that strictly adheres to system dynamics and state-action constraints. This framework provides two crucial advantages. First, it accelerates learning by restricting the search to the space of valid trajectories. Second, it enables safe training directly on physical systems, as constraint satisfaction is guaranteed even for exploratory actions during the learning process.

Experimental results validate the approach on a range of problems, from a linear system with convex constraints to a nonlinear kinematic bicycle model with nonconvex exclusion zones. The results confirm that the layer successfully enforces all constraints while providing meaningful gradients for learning, highlighting the method's practical utility for developing verifiably safe learning-based controllers.

Contents

Abstract iii

List of Figures vii

1 Introduction 1

2 Notation 3
2.1 Differentiation . . . 3
2.2 Other . . . 4

3 Literature Review 5
3.1 Projection Layers . . . 5
3.2 Automatic Differentiation . . . 6
3.3 Implicit Function Differentiation . . . 7
3.3.1 Computing the IFT-VJP: . . . 7
3.4 Sequential Quadratic Programming (SQP) . . . 8
3.5 Operator Splitting Solver for Quadratic Programs . . . 10
3.6 Explicit MPC . . . 12
3.7 Predictive Safety Filter . . . 13

4 SQP Layer 15
4.1 Optimization Problem Formulation . . . 15
4.1.1 Positive Definite Hessian Estimate . . . 15
4.1.2 Quadratic programming subproblem . . . 17
4.1.3 Line search . . . 17
4.1.4 Convergence Criteria . . . 18
4.2 Backward Pass . . . 18
4.3 Algorithm Summary . . . 20

5 Rollout Projection 23
5.1 Control Problem . . . 23
5.2 Learning Approach . . . 23
5.3 SQP Projection Layer . . . 23
5.4 Loss Function . . . 24
5.5 Training Procedure . . . 25

6 Results 27
6.1 Linear System - Convex Constraints . . . 27
6.2 Linear System - Nonconvex Constraints . . . 28
6.3 Nonlinear System with Nonconvex Constraints . . . 30
6.4 Nonconvex Clover Projection . . . 31

7 Conclusion **35**

Bibliography **37**

List of Figures

5.1 Receding Horizon Network Pipeline. The alteration shown in orange allows the projection cost to have meaningful gradients even if the projection hasn't met the convergence criteria. 24

6.1 Comparison of raw and projected rollouts for the linear system with binding state and action constraints. 27
6.2 The VJP relates output perturbations, here an indicator at s_4, to input perturbations. We see that binding constraints predetermine the projection output and eliminate sensitivity with respect to parts of the input. 28
6.3 Projection with nonconvex constraint: all discrete states s_i satisfy the exclusion zone. 28
6.4 Forward pass verification: residual evolution with $\epsilon_{\text{SQP}} = 10^{-5}$ and $\epsilon_{\text{OSQP}} = 10^{-6}$. 29
6.5 Backward pass verification: comparison of VJP against finite differences for nonconvex system. The x-axis shows the number of CG iterations used to solve (4.10e). Convergence is achieved after about 100 iterations, with a slight underestimation due to the regularization that makes constraints "appear less active". 29
6.6 Illustration of kinematic bike model. 30
6.7 Comparison of raw and projected coordinate trajectories for the bike system. . . 31
6.8 Difference between the bike rollout proposal and the projected trajectory in states d_s and actions d_a. 31
6.9 Forward pass verification: nonconvex constraints, nonlinear bike system rollout projection with $\epsilon_{\text{SQP}} = 10^{-2}$ and $\epsilon_{\text{OSQP}} = 10^{-3}$. 32
6.10 Backward pass verification: comparison of VJP against finite differences for the bike system. The x-axis indicates the number of CG iterations used to solve (4.10e). 32
6.11 Nonconvex projection onto the clover set. Neighboring points may be projected into different leaves, highlighting the inherent nonconvexity of the problem. . . . 33
6.12 Histogram of forward $\langle \text{JVP}(v), u \rangle$ and reverse $\langle v, \text{VJP}(u) \rangle$ differences. After filtering out discontinuous regions, the VJP closely matches the finite-difference JVP. The line of zero valued VJP are problems that don't meet convergence citeria. 33
6.13 Finite-difference JVP failure cases. At discontinuities, the two evaluations are projected onto different clover leaves. 34

Chapter 1

Introduction

Neural networks have become a powerful tool for modeling complex systems, but they are typically trained with *soft constraints*, where penalty terms in the loss function discourage but do not guarantee constraint satisfaction [18]. In safety-critical domains such as autonomous driving, robotics, and industrial control, this limitation poses unacceptable risks. A single violation of safety constraints can lead to equipment damage, mission failure, or catastrophic accidents. For these applications, it is not enough to merely encourage constraint satisfaction; mathematical guarantees are essential.

Recent advances in deep learning have introduced *implicit layers* [1, 13], which define their outputs through optimization problems rather than fixed computation graphs. This paradigm enables the direct integration of physical laws, safety conditions, and domain-specific knowledge into network architectures, thereby facilitating seamless integration of these elements. Crucially, implicit layers open the door to enforcing *hard constraints* during both inference and training, ensuring that learned models always remain within feasible regions. However, while effective methods exist for affine and convex constraints [17], many real-world systems require handling *nonlinear and nonconvex* constraint sets. Examples include obstacle avoidance in robotics, where feasible paths must exclude irregular, nonconvex regions, and physical systems governed by nonlinear dynamics.

Addressing nonconvex constraints remains a significant challenge. Existing methods highlight both the promise and the limitations of current approaches. The DC3 framework integrates equality and inequality constraints through a differentiable completion-and-correction procedure, showing that deep learning architectures can be augmented to handle constrained nonconvex settings [14]. More recently, the ENFORCE framework introduced adaptive-depth neural projections, demonstrating how local linearization can enable scalable training with nonlinear equality constraints [19]. Together, these approaches represent essential steps toward embedding nonconvex optimization principles into neural networks.

This thesis builds on these ideas by introducing a novel *nonconvex projection layer* based on Sequential Quadratic Programming (SQP). Unlike purely first-order correction or linearization approaches, the SQP formulation leverages second-order curvature information to achieve faster and more reliable convergence. Each iteration solves a quadratic subproblem efficiently via operator splitting quadratic programming (OSQP), while a merit function with line search provides globalization, ensuring robust convergence even when neural network proposals are far from feasible. Differentiability is achieved via the Implicit Function Theorem (IFT). By formulating constraint enforcement as a Euclidean projection problem, the SQP layer aims to map inputs to the nearest feasible point, preserving interpretability and providing a principled notion of "minimal intervention." At the same time, the diagonal Hessian approximation strikes a balance between the benefits of second-order methods and computational tractability, making the approach suitable for high-dimensional learning tasks.

The objectives of this thesis are:

1. Develop a rigorous SQP-based framework for enforcing hard nonconvex constraints in neural networks through parametric optimization.

2. Derive exact, stable gradients using the Implicit Function Theorem to enable end-to-end training.

3. Demonstrate applicability in control systems through rollout projection, where safety-critical guarantees are essential.

4. Validate the approach on problems of increasing complexity, from convex linear systems to nonlinear systems with nonconvex constraints.

The remainder of the thesis is structured as follows. Chapter 3 introduces the mathematical notation and foundations. Chapter 4 reviews projection layers, automatic differentiation, implicit function differentiation, and sequential quadratic programming. Chapter 5 presents the SQP layer formulation, quadratic subproblem structure, and backward pass derivation. Chapter 6 demonstrates the method of rollout projection for control. Chapter 7 reports experimental validation across systems of increasing complexity. The thesis concludes with Chapter 8, which presents an analysis of the contributions, limitations, and future directions.
By combining second-order optimization principles with line search globalization, this work advances beyond existing frameworks to provide a robust, efficient, and interpretable mechanism for enforcing hard nonconvex constraints. In doing so, it opens new possibilities for deploying neural networks in high-stakes domains where constraint violations are fundamentally unacceptable.

Chapter 2

Notation

2.1 Differentiation

Let $h(x, \phi)$ be a vector-valued function

$$h : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^p. \tag{2.1}$$

The Jacobian of h with respect to x, evaluated at $(\hat{x}, \hat{\phi})$, is

$$J_{h,x}(\hat{x}, \hat{\phi}) = \left(\frac{\partial h(x, \phi)}{\partial x} \right)_{(\hat{x}, \hat{\phi})} \in \mathbb{R}^{p \times n}. \tag{2.2}$$

Given $y \in \mathbb{R}^p$, the vector-Jacobian product is

$$y^\top J_{h,x}(\hat{x}, \hat{\phi}) = \left(\frac{\partial (y^\top h(x, \phi))}{\partial x} \right)_{(\hat{x}, \hat{\phi})} = y^\top \left(\frac{\partial h(x, \phi)}{\partial x} \right)_{(\hat{x}, \hat{\phi})} \in \mathbb{R}^{1 \times n}. \tag{2.3}$$

The vector-Hessian product is

$$y^\top J_{h,xx}(\hat{x}, \hat{\phi}) = \left(\frac{\partial^2 (y^\top h(x, \phi))}{\partial x^2} \right)_{(\hat{x}, \hat{\phi})} = y^\top \left(\frac{\partial^2 h(x, \phi)}{\partial x^2} \right)_{(\hat{x}, \hat{\phi})} \in \mathbb{R}^{n \times n}, \tag{2.4}$$

where $J_{h,xx}(\hat{x}, \hat{\phi})$ is the Hessian tensor of h (shape $p \times n \times n$).
The vector-mixed derivative product is

$$y^\top J_{h,x\phi}(\hat{x}, \hat{\phi}) = \left(\frac{\partial^2 (y^\top h(x, \phi))}{\partial \phi \partial x} \right)_{(\hat{x}, \hat{\phi})} = y^\top \left(\frac{\partial^2 h(x, \phi)}{\partial \phi \partial x} \right)_{(\hat{x}, \hat{\phi})} \in \mathbb{R}^{m \times n}, \tag{2.5}$$

where $J_{h,x\phi}(\hat{x}, \hat{\phi})$ is the mixed derivative tensor (shape $p \times m \times n$) with subscripts indicating differentiation order from left to right. The transpose relation

$$\left(y^\top J_{h,x\phi}(\hat{x}, \hat{\phi}) \right)^\top = J_{h,\phi x}(\hat{x}, \hat{\phi}) y \in \mathbb{R}^{n \times m} \tag{2.6}$$

holds when h is twice continuously differentiable. Schwarz's theorem ensures that mixed partial derivatives are equal:

$$J_{h,x\phi}(\hat{x}, \hat{\phi}) = J_{h,\phi x}(\hat{x}, \hat{\phi}), \tag{2.7}$$

where the equality is understood in the sense that the (i, j, k)-th component of the left tensor equals the (i, k, j)-th component of the right tensor.

2.2 Other

- We denote $\mathrm{D} : \mathbb{R}^n \to \mathbb{R}^{n \times n}$ as the diagonal operator mapping a vector to a square matrix that is all zeros except for this vector on its diagonal.
- The indicator function of a set $\mathcal{C}$ is $\Psi_{\mathcal{C}}(z) = 0$ if $z \in \mathcal{C}$, and $+\infty$ otherwise.
- $\Pi_{[l,u]}(\cdot)$ denotes the projection operator onto the box constraints $[l, u]$.

Chapter 3

Literature Review

3.1 Projection Layers

Hard-constrained neural networks represent a paradigm shift from traditional unconstrained deep learning architectures by incorporating mathematical constraints directly into the network. These constraints encode prior domain knowledge that a network's output *must* satisfy to be considered valid, like physical laws [14]. Traditionally, most deep learning models have relied on "soft" constraints [18], which use penalty terms in a loss function to discourage constraint violations. However, this approach offers no guarantee of satisfaction, posing an unacceptable risk in safety-critical and industrial applications.
Hard-constrained neural networks are characterized by containing implicit layers. In contrast to classical explicit layers, which follow an explicit computation graph from input to output, such as sequentially applying learned transformations and activations, an implicit layer defines the output implicitly by the relationship that an input-output pair must satisfy. The forward pass output for a given input is then obtained as a solution to a parametric optimization problem, typically computed by an iterative solver. The backward pass through these layers requires computing the derivatives of the solution to this optimization problem with respect to its input parameters, typically achieved through implicit differentiation.
To make this statement more formal, we define a generic Euclidean projection layer $f : \mathbb{R}^n \to \mathbb{R}^n$ where, for the sake of simplicity, the input $x \in \mathbb{R}^n$ and output $y \in \mathbb{R}^n$ are of the same dimension:

$$\begin{aligned} y^* = \min_{y \in \mathbb{R}^n} \ & \|x - y\|_2^2 \\ \text{s.t. } & y \in \mathcal{C}. \end{aligned} \tag{3.1}$$

There exist various types of projection layers, characterized by the class of constraints they allow:

1. **Affine Constraints** With $\mathcal{C}$ representing a polytope, affine constraints represent the most elemental form of constraints. Affine constraints turn (3.1) directly into solving an efficient quadratic program [2] or, when considering non-Euclidean projection, even to closed form [20].

2. **Convex Constraints** With $\mathcal{C}$ representing a convex set, (3.1) remains in the class of convex optimization problems. The approaches tackling such constraints, therefore, either leverage convex optimization directly [1], use ray projection onto the convex set [24], or leverage operator splitting approaches [17].

3. **Non Convex Constraints** In the most general case, where the constraints $\mathcal{C}$ are non-convex, approaches typically leverage iterative refinement. Two notable examples are [19],

which uses successive constraint linearizations, and [14], which uses, among other things, an inner violation rectification gradient descent routine.

3.2 Automatic Differentiation

In modern machine learning, automatic differentiation (AD) has become the cornerstone for efficiently computing gradients [10, 16]. Automatic differentiation decomposes complex functions into elementary operations, each with known derivatives, and applies the chain rule systematically. There are two primary modes of AD: *forward mode* and *reverse mode*.

Forward mode (Jacobian-vector product, JVP) Forward mode computes derivatives by propagating tangent (derivative) information alongside values through the computational graph. Given $f : \mathbb{R}^n \to \mathbb{R}^m$ and a vector $u \in \mathbb{R}^n$, it computes the Jacobian-vector product (JVP)

$$J_f(x)\, u \in \mathbb{R}^m,$$

where $J_f(x) \in \mathbb{R}^{m \times n}$ is the Jacobian of f at x. This is efficient when n (the number of inputs) is small. Intuitively, the JVP is a direct generalization of the directional derivative, indicating how the output changes for a small input change in direction u.

Reverse mode (vector-Jacobian product, VJP) Reverse mode, also known as backpropagation, computes the vector-Jacobian product (VJP) instead. Given $v \in \mathbb{R}^m$, it evaluates

$$v^\top J_f(x) \in \mathbb{R}^n,$$

by propagating sensitivities (adjoints) backward from outputs to inputs. This is efficient when m (the number of outputs) is small, which is the typical case in machine learning where f is a scalar loss function. Intuitively, the VJP indicates how the input needs to change to result in a small change in the output in direction v.

Example Consider a scalar-valued function that is the composition of simpler functions:

$$y = f(x) = f_3(f_2(f_1(x))),$$

where

$$f_1(x) = Wx, \quad f_2(z) = \sigma(z), \quad f_3(h) = v^\top h,$$

with $W \in \mathbb{R}^{k \times n}$, $\sigma(\cdot)$ a nonlinear activation function applied elementwise, and $v \in \mathbb{R}^k$.

- **Forward pass (values):**

$$z = f_1(x) = Wx, \quad h = f_2(z) = \sigma(z), \quad y = f_3(h) = v^\top h.$$

- **Backward pass (reverse mode AD, VJP):** Starting with the adjoint $\bar{y} = \frac{dy}{dy} = 1$, we propagate row-vector adjoints backward using VJPs:

$$\bar{h}^\top = \bar{y}^\top J_{f_3}(h) = 1^\top \cdot v^\top \in \mathbb{R}^{1 \times k},$$

$$\bar{z}^\top = \bar{h}^\top J_{f_2}(z) = v^\top \operatorname{diag}(\sigma'(z)) \in \mathbb{R}^{1 \times k},$$

$$\bar{x}^\top = \bar{z}^\top J_{f_1}(x) = \big(v^\top \operatorname{diag}(\sigma'(z))\big) W \in \mathbb{R}^{1 \times n}.$$

  Each step is a VJP of the form:

$$\bar{z}^\top = \bar{h}^\top J_{f_2}(z),$$

  and the final result $\bar{x} \in \mathbb{R}^n$ is the gradient of y with respect to x.

3.3 Implicit Function Differentiation

Not all layers admit a closed-form representation to obtain an output $y \in \mathbb{R}^m$ from an input $x \in \mathbb{R}^n$. Implicit layers instead define the output through the relation:

$$K : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^m, \quad K(x, y) = 0,$$

which any valid input-output pair (x, y) must satisfy. Suppose there exists a function:

$$k : \mathbb{R}^n \to \mathbb{R}^m, \quad k(x) = y,$$

such that $K(x, k(x)) = 0$. Then k constitutes the desired *implicit function*.
In this setting, standard automatic differentiation (cf. Section 3.2) cannot be applied directly: the explicit computation graph for k may be unavailable, or unrolling an iterative solver can be both numerically expensive and unstable. Instead, the *implicit function theorem* [13] (IFT) provides an efficient and stable alternative for computing derivatives of k with respect to x.

Demonstration of the implicit function theorem (IFT) Suppose that at a point (x, y) solving $K(x, y) = 0$, the Jacobian

$$J_{K,y}(x, y) \in \mathbb{R}^{m \times m}$$

is invertible. By the IFT, there exists locally a function $k(x)$ with $K(x, k(x)) = 0$. Differentiating this identity with respect to x using the multivariable chain rule yields

$$\frac{d}{dx} K\big(x, k(x)\big) \;=\; J_{K,x}(x, y) \;+\; J_{K,y}(x, y)\, J_{k,x}(x) \;=\; 0.$$

Rearranging and solving for $J_{k,x}(x)$ gives the standard formula

$$\boxed{J_{k,x}(x) \;=\; -J_{K,y}(x, y)^{-1}\, J_{K,x}(x, y).} \tag{3.2}$$

Example Consider $K(x, y) = y^2 - x = 0$, with $x, y \in \mathbb{R}$. We compute

$$J_{K,y}(x, y) = 2y, \quad J_{K,x}(x, y) = -1.$$

By the IFT formula,

$$J_{k,x}(x) = -J_{K,y}(x, y)^{-1} J_{K,x}(x, y) = -(2y)^{-1} \cdot (-1) = \tfrac{1}{2y}.$$

For the positive root $y = \sqrt{x}$, this matches the derivative of the implicit function $k(x) = \sqrt{x}$.

3.3.1 Computing the IFT-VJP:

Let $v \in \mathbb{R}^m$ be the adjoint (a row vector in the backward pass) that we want to propagate through the implicit layer, meaning we seek the VJP

$$w^\top = v^\top J_{k,x}(x).$$

Utilizing the IFT (3.2) we have,

$$v^\top J_{k,x}(x) = -v^\top J_{K,y}(x, y)^{-1}\, J_{K,x}(x, y).$$

Step 1: introduce an auxiliary variable Define

$$u^\top = -v^\top J_{K,y}(x,y)^{-1}.$$

Then the VJP becomes

$$w^\top = u^\top J_{K,x}(x,y).$$

Step 2: avoid explicit inversion By rearranging the definition of u, we get

$$u^\top J_{K,y}(x,y) = -v^\top,$$

or equivalently

$$J_{K,y}(x,y)^\top u = -v.$$

Thus u can be obtained by solving the linear system

$$J_{K,y}(x,y)^\top u = -v. \tag{3.3}$$

Step 3: final expression Once u is found, the desired VJP is obtained as

$$w^\top = u^\top J_{K,x}(x,y). \tag{3.4}$$

Summary of variables

- $v \in \mathbb{R}^m$: incoming adjoint (from the downstream gradient).
- $u \in \mathbb{R}^m$: auxiliary vector obtained by solving (3.3).
- $w \in \mathbb{R}^n$: resulting adjoint with respect to the input x, given by (3.4).

This formulation avoids ever forming the inverse $J_{K,y}(x,y)^{-1}$ explicitly. It requires only the ability to compute vector-Jacobian products and to solve the linear system involving $J_y K(x,y)$ or its transpose.

3.4 Sequential Quadratic Programming (SQP)

Sequential quadratic programming (SQP) methods provide one of the most powerful frameworks for solving nonlinear constrained optimization problems [3]. Consider the nonlinear program

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & f(x) && \text{(3.5a)} \\ \text{s.t.} \quad & h(x) = 0, \quad h : \mathbb{R}^n \to \mathbb{R}^{m_e}, && \text{(3.5b)} \\ & g(x) \le 0, \quad g : \mathbb{R}^n \to \mathbb{R}^{m_i}, && \text{(3.5c)} \end{aligned}$$

with m_e equality constraints and m_i inequality constraints, and the accompanying Lagrangian

$$\mathcal{L}(x,\lambda,\mu) = f(x) + \lambda^\top h(x) + \mu^\top g(x). \tag{3.6}$$

KKT conditions For a point $x^\star \in \mathbb{R}^n$ to be considered a local solution of (3.5), there must exist multipliers $\lambda^\star \in \mathbb{R}^{m_e}$ (for equalities) and $\mu^\star \in \mathbb{R}^{m_i}$ (for inequalities) satisfying the Karush–Kuhn–Tucker (KKT) [9] conditions:

1. **Stationarity:** $\nabla f(x^\star) + J_h(x^\star)^\top \lambda^\star + J_g(x^\star)^\top \mu^\star = 0.$
2. **Primal feasibility:** $h(x^\star) = 0, \quad g(x^\star) \leq 0.$
3. **Dual feasibility:** $\mu^\star \geq 0.$
4. **Complementary slackness:** $\mathrm{D}(\lambda^*)h(x^\star) = 0, \quad \mathrm{D}(\mu^\star)\, g(x^\star) = 0$

Here $J_h(x)$ and $J_g(x)$ denote the Jacobians of the equality and inequality constraints, respectively.

Core Idea Starting from an initial "guess" $p^0 = [x^0, \lambda^0, \mu^0]$, SQP operates under the principle of iterative refinement to obtain a solution p^* satisfying the KKT conditions. Or in other words, reducing their violation, referred to as residuals.

Quadratic programming subproblem At the current iterate p^k, the Lagrangian is approximated quadratically in x, while the constraints are linearized [3]. This yields the QP subproblem

$$\begin{aligned} \min_{d_x \in \mathbb{R}^n} \quad & \tfrac{1}{2} d_x^\top B^k d_x + \nabla f(x^k)^\top d_x && \text{(3.7a)} \\ \text{s.t.} \quad & h(x^k) + J_h(x^k) d_x = 0, && \text{(3.7b)} \\ & g(x^k) + J_g(x^k) d_x \leq 0, && \text{(3.7c)} \end{aligned}$$

where $B^k \succ 0$ is a positive definite approximation of the Lagrangian Hessian $J_{\mathcal{L},xx}(x^k, \lambda^k, \mu^k)$. This approximation is required for (3.7) to be always well defined and guarantees a unique solution. Without positive definiteness, the QP may become indefinite or unbounded below. In such cases, no meaningful search direction d_x^k can be extracted, and the algorithm can stall.

Iterative update The solution $(d_x^k, \hat{\lambda}^k, \hat{\mu}^k)$ of (3.7) provides a search direction for the primal and dual variables. With step size β^k determined by a line search, the iterates are updated as

$$x^{k+1} = x^k + \beta^k d_x^k, \quad \lambda^{k+1} = \lambda^k + \beta^k(\hat{\lambda}^k - \lambda^k), \quad \mu^{k+1} = \mu^k + \beta^k(\hat{\mu}^k - \mu^k).$$

Line search While the Newton-type steps of SQP are locally effective, they require a globalization strategy to guarantee convergence from poor initializations [3]. A standard approach is to employ a merit function that balances progress toward reducing the objective with progress toward satisfying the constraints.
One such choice, the ℓ_1-merit function, is defined by:

$$\phi(x; \rho) = f(x) \; + \; \rho\big(\|h(x)\|_1 + \|\max\{0, g(x)\}\|_1\big), \tag{3.8}$$

where $\rho > 0$ is a penalty parameter. Here, $\|h(x)\|_1$ measures violation of equality constraints, and $\|\max\{0, g(x)\}\|_1$ measures violation of inequality constraints. The penalty parameter ρ is chosen sufficiently large to ensure that any decrease in ϕ implies progress toward satisfying the KKT conditions. To ensure convergence from any starting point, a line search strategy is applied along the SQP search direction d_x^k. Specifically, the step size $\beta^k \in (0, 1]$ is chosen via binary search to satisfy the sufficient decrease (Armijo-type [3, 4]) condition:

$$\phi(x^k + \beta^k d_x^k; \rho) \; \leq \; \phi(x^k; \rho) + \eta \beta^k \nabla \phi(x^k; \rho)^\top d_x^k,$$

with a parameter $\eta \in (0, 1)$ chosen sufficiently small, typically $\eta = 10^{-4}$ [21]. This guarantees that every accepted step reduces the merit function, yielding convergence to a KKT point, even when the solution is far away.

Remarks SQP thus reduces the original nonlinear constrained problem to a sequence of structured QP subproblems. Its main strengths are:

- *Second-order accuracy:* via B^k approximating the Hessian of $\mathcal{L}$.
- *Constraint handling:* via local linearizations of $h(x)$ and $g(x)$.
- *Practical efficiency:* quasi-Newton updates (e.g., BFGS [15]) avoid forming $\nabla^2_{xx}\mathcal{L}$ explicitly.

This balance of theoretical rigor and numerical robustness has made SQP the workhorse of nonlinear constrained optimization.

3.5 Operator Splitting Solver for Quadratic Programs

This section presents the Operator Splitting Quadratic Program (OSQP) algorithm [22, 23], a popular and efficient method for solving problems of the form:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \ & \tfrac{1}{2} x^\top Q x + c^\top x \\ \text{s.t. } & l \leq Ax \leq u, \end{aligned}$$

given $Q \in \mathbb{S}_+$, which we will restrict to the case of our application $Q = \mathrm{D}(q)$ with $q \geq 0$.

Problem Reformulation and Augmented Lagrangian The OSQP algorithm begins by reformulating the constrained QP through variable splitting. We introduce auxiliary variables $\tilde{x}$ and $\tilde{z}$ to decouple the objective function from the constraints:

$$\begin{aligned} \min_{x, \tilde{x} \in \mathbb{R}^n, z, \tilde{z} \in \mathbb{R}^p} \ & \frac{1}{2} \tilde{x}^\top \mathrm{D}(q) \tilde{x} + c^\top \tilde{x} + \Psi_{[l,u]}(z) + \Psi_{Ax=z}(\tilde{x}, \tilde{z}) \\ \text{subject to } & (\tilde{x}, \tilde{z}) = (x, z). \end{aligned} \tag{3.9}$$

The corresponding augmented Lagrangian formulation incorporates penalty terms to handle the equality constraints:

$$\begin{aligned} \mathcal{L}(\tilde{x}, x, \tilde{z}, z, w, y) = & \frac{1}{2} \tilde{x}^\top \mathrm{D}(q) \tilde{x} + c^\top \tilde{x} + \Psi_{[l,u]}(z) + \Psi_{Ax=z}(\tilde{x}, \tilde{z}) \\ & + w^\top (\tilde{x} - x) + \frac{\sigma}{2} \|\tilde{x} - x\|^2 \\ & + y^\top (\tilde{z} - z) + \frac{\rho}{2} \|\tilde{z} - z\|^2, \end{aligned} \tag{3.10}$$

where w and y are dual variables, and $\sigma > 0$ and $\rho > 0$ are penalty parameters.

ADMM Update Equations The Alternating Direction Method of Multipliers [8] (ADMM) iteratively minimizes the augmented Lagrangian with respect to different variable blocks. The resulting update equations are:

$$(\tilde{x}^{k+1}, \tilde{z}^{k+1}) = \underset{\tilde{x},\tilde{z}}{\operatorname{argmin}}\ \mathcal{L}(\tilde{x}, x^k, \tilde{z}, z^k, w^k, y^k), \tag{3.11a}$$

$$x^{k+1} = \underset{x}{\operatorname{argmin}}\ \mathcal{L}(\tilde{x}^{k+1}, x, \tilde{z}^{k+1}, z^k, w^k, y^k), \tag{3.11b}$$

$$z^{k+1} = \underset{z}{\operatorname{argmin}}\ \mathcal{L}(\tilde{x}^{k+1}, x^{k+1}, \tilde{z}^{k+1}, z, w^k, y^k), \tag{3.11c}$$

$$w^{k+1} = w^k + \sigma(\tilde{x}^{k+1} - x^{k+1}), \tag{3.11d}$$

$$y^{k+1} = y^k + \rho(\tilde{z}^{k+1} - z^{k+1}). \tag{3.11e}$$

Explicit Update Steps The first subproblem (3.11a) is solved by substituting the constraint $A\tilde{x} = \tilde{z}$ directly into the augmented Lagrangian (3.10), yielding an unconstrained optimization problem in $\tilde{x}$. Setting the gradient to zero provides a linear system of equations. Since its LHS is positive definite, we employ conjugate gradient descent (CG) to obtain $\tilde{x}^{k+1}$. This approach is called the indirect method [23]. The complete set of explicit update equations becomes:

$$\begin{aligned}(\mathrm{D}(q) + \sigma I + A^\top \rho A)\tilde{x}^{k+1} &= \sigma x^k - c - w^k + A^\top(\rho z^k - y^k) \quad \textit{Solve for } \tilde{x}^{k+1} \textit{ using CG.}\\ \tilde{z}^{k+1} &= A\tilde{x}^{k+1}\end{aligned} \tag{3.12a}$$

$$x^{k+1} = \tilde{x}^{k+1} + \sigma^{-1} w^k \tag{3.12b}$$

$$z^{k+1} = \Pi_{[l,u]}(\tilde{z}^{k+1} + \rho^{-1} y^k) \tag{3.12c}$$

$$w^{k+1} = w^k + \sigma(\tilde{x}^{k+1} - x^{k+1}) \tag{3.12d}$$

$$y^{k+1} = y^k + \rho(\tilde{z}^{k+1} - z^{k+1}). \tag{3.12e}$$

Note that the dual variables y^k can be decomposed as $y^k = y_+^k - y_-^k$, where $y_+^k \geq 0$ and $y_-^k \geq 0$ correspond to the upper and lower bound constraints, respectively. This decomposition reflects the KKT conditions: $y^k > 0$ when upper bounds are active, $y^k < 0$ when lower bounds are active. Another important observation is that substituting equation (3.12b) into (3.12d) yields $w^{k+1} = 0$ for all $k \in \mathbb{N}_+$, effectively eliminating the dual variable w from all subsequent iterations, as noted in [23].

Relaxation and Penalty Matrix OSQP incorporates over-relaxation through the parameter $\alpha \in (0, 2)$ to improve convergence properties. Additionally, the scalar penalty parameter ρ is generalized to a diagonal penalty matrix $R \in \mathbb{S}_{++}^p$ to handle constraints with different characteristics:

$$(\mathrm{D}(q) + \sigma I + A^\top R A)\tilde{x}^{k+1} = \sigma x^k - c + A^\top(R z^k - y^k) \tag{3.13a}$$

$$\tilde{z}^{k+1} = A\tilde{x}^{k+1} \tag{3.13b}$$

$$x^{k+1} = \alpha\tilde{x}^{k+1} + (1-\alpha)x^k \tag{3.13c}$$

$$z^{k+1} = \Pi_{[l,u]}(\alpha\tilde{z}^{k+1} + (1-\alpha)z^k + R^{-1}y^k) \tag{3.13d}$$

$$y^{k+1} = y^k + R(\alpha\tilde{z}^{k+1} + (1-\alpha)z^k - z^{k+1}) \tag{3.13e}$$

The penalty matrix R is constructed to distinguish between equality and inequality constraints [22]:

$$R = \mathrm{diag}(\rho) \quad \text{where} \quad \rho_i = \begin{cases} \bar{\rho}_i & \text{if} \quad l_i \neq u_i \quad \text{(inequality constraint)} \\ 10^3 \cdot \bar{\rho}_i & \text{if} \quad l_i = u_i \quad \text{(equality constraint)} \end{cases} \tag{3.14}$$

with $\bar{\rho}_i > 0$. This weighting strategy assigns higher penalty values to equality constraints, ensuring their stricter enforcement.

Convergence Criteria OSQP achieves convergence when the primal and dual variables satisfy the KKT optimality conditions. Due to the reformulated structure in (3.9), which eliminates explicit inequality constraints through the projection operator, dual feasibility is automatically maintained throughout the algorithm's execution. Furthermore, complementary slackness conditions are satisfied by construction at machine precision through ADMM's projection-based updates [23].
Consequently, convergence verification reduces to monitoring only two of the four KKT conditions: stationarity (*dual residual*) and primal feasibility (*primal residual*). The algorithm terminates when both residuals satisfy their respective tolerance criteria:

$$\|r^k_{\text{prim}}\|_\infty \leq \epsilon_{\text{prim}}, \quad \text{where } r^k_{\text{prim}} = Ax^k - z^k \tag{3.15a}$$

$$\|r^k_{\text{dual}}\|_\infty \leq \epsilon_{\text{dual}}, \quad \text{where } r^k_{\text{dual}} = \mathrm{D}(q)x^k + q + A^\top y^k \tag{3.15b}$$

Adaptive Penalty Parameter Updates To ensure balanced convergence between the primal and dual iterates, OSQP incorporates an adaptive penalty adjustment mechanism [8, 26]. The penalty parameter ρ is dynamically updated based on the relative magnitudes of the normalized residuals to prevent one type of residual from dominating the convergence behavior:

$$\rho^{k+1} = \begin{cases} 2\rho^k & \text{if } \left\|r^k_{\text{prim}}/\epsilon_{\text{prim}}\right\|_2^2 \geq 10^2 \left\|r^k_{\text{dual}}/\epsilon_{\text{dual}}\right\|_2^2 \\ \frac{1}{2}\rho^k & \text{if } \left\|r^k_{\text{dual}}/\epsilon_{\text{dual}}\right\|_2^2 \geq 10^2 \left\|r^k_{\text{prim}}/\epsilon_{\text{prim}}\right\|_2^2 \\ \rho^k & \text{otherwise.} \end{cases} \tag{3.16}$$

This adaptive strategy increases the penalty parameter when primal residuals significantly exceed dual residuals (promoting dual feasibility improvement) and decreases it when the dual residuals dominate (encouraging primal feasibility improvement). Such automatic balancing enhances algorithmic robustness and eliminates the need for manual tuning of the penalty parameter across diverse problem instances and linearizations encountered in sequential quadratic programming applications.

3.6 Explicit MPC

Explicit model predictive control (EMPC) replaces online optimization with offline precomputation by expressing the optimal control law as a piecewise affine (PWA) function of the state. The foundational work by Bemporad et al. [6] demonstrated that linear MPC with convex constraints yields a PWA control law $u^*(x) = F_i x + G_i$ defined over polyhedral regions $\mathcal{R}_i$, each corresponding to a different active constraint set. This enables real-time implementation through simple function evaluations rather than quadratic programming.
However, explicit MPC faces significant scalability challenges due to exponential growth in polyhedral regions with problem dimension [7]. This has motivated the development of approximate methods, including neural network approximations [11], which can compactly represent complex control laws while reducing memory requirements.
A critical limitation of many approximation approaches is the lack of *constraint satisfaction guarantees*. While neural networks can effectively approximate the optimal control law, they may produce infeasible controls that violate system constraints, which are a fundamental requirement in safety-critical applications. This challenge motivates the development of methods that combine the computational efficiency of approximate explicit MPC with rigorous constraint handling capabilities.

3.7 Predictive Safety Filter

The predictive safety filter (PSF) framework provides a modular approach to ensuring constraint satisfaction for arbitrary control policies by solving a minimal-intervention optimization problem in real-time [25]. PSF employs model predictive control principles to anticipate future constraint violations and preemptively modify unsafe control inputs at the *current* timestep. Given a potentially unsafe desired control input u_d, the PSF computes a safe control input u_s by solving:

$$u_s = \arg\min_u \|u - u_d\|^2 \quad \text{subject to predicted constraint satisfaction} \tag{3.17}$$

The key insight of Wabersich and Kohler [25] is that this minimal-intervention principle allows the integration of safety guarantees with high-performance but potentially unsafe controllers, including learning-based policies. The PSF framework is particularly valuable in reinforcement learning applications, where exploration-based policies may generate actions that violate constraints during training or deployment.

Chapter 4

SQP Layer

4.1 Optimization Problem Formulation

We focus on quadratic nonlinear parametric constrained problems of the form

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \ & \tfrac{1}{2} x^\top \mathrm{D}(q) x + c^\top x \\ \text{s.t. } & l \leq h(x, \phi) \leq u, \end{aligned} \tag{4.1}$$

where $x \in \mathbb{R}^n$ is the optimization variable. The objective is determined by $q \in \mathbb{R}^n_{\geq 0}$ and $c \in \mathbb{R}^n$. The constraints are defined by the function $h : \mathbb{R}^n \times \mathbb{R}^m \mapsto \mathbb{R}^p$, the constraint parameter $\phi \in \mathbb{R}^m$ and bounds $l, u \in \mathbb{R}^p$. Notice that the bounds l, u are non-parametric and ϕ only interacts with h. As $h(x, \phi)$ may be nonlinear, problem (4.1) is, in general, nonconvex. We introduce two dual variables $y_+ \in \mathbb{R}^p$ for the upper constraint, and $y_- \in \mathbb{R}^p$ for the lower constraint. The Lagrangian $\mathcal{L}(x, y_+, y_-)$ is then:

$$\mathcal{L}(x, y_+, y_-) = \frac{1}{2} x^\top \mathrm{D}(q) x + c^\top x + y_+^\top (h(x, \phi) - u) + y_-^\top (l - h(x, \phi)). \tag{4.2}$$

The first order optimality conditions [9] of this primal-dual problem are:

Stationarity:	$\mathrm{D}(q)x + c + J_{h,x}(x, \phi)^\top (y_+ - y_-) = 0,$	(4.3a)
Primal feasibility:	$l \leq h(x, \phi) \leq u,$	(4.3b)
Dual feasibility:	$y_+ \geq 0$ and $y_- \geq 0,$	(4.3c)
Complementary slackness:	$D(y_+)(h(x, \phi) - u) = 0$ and $D(y_-)(l - h(x, \phi)) = 0.$	(4.3d)

To solve problem (4.1), we will leverage SQP (Section 3.4) to find a point $p^* = \begin{bmatrix} x & y_+ & y_- \end{bmatrix}$ satisfying the KKT conditions.

4.1.1 Positive Definite Hessian Estimate

Let us begin by calculating the exact Hessian of the Lagrangian with respect to x:

$$J_{\mathcal{L},xx}(x^k, y_+^k, y_-^k) = \mathrm{D}(q) + (y_+^k - y_-^k)^\top J_{h,xx}(x^k, \phi). \tag{4.4}$$

If h is linear, then $J_{h,xx}(x^k, \phi) = 0$, and (4.4) reduces to $\mathrm{D}(q)$, which is positive semi-definite by convention. For nonlinear h, however, the second term introduces curvature in directions determined by $y_+^k - y_-^k$ and $J_{h,xx}$, with no guarantee that the overall matrix remains positive semidefinite.

Approximation strategy In practice, the exact Hessian (4.4) is replaced by a *positive definite approximation* $B^k \succ 0$. Our approach is to (i) extract only the diagonal entries of the curvature term $(y_+^k - y_-^k)^\top J_{h,xx}(x^k, \phi)$, and (ii) enforce positive definiteness by *clipping* the diagonal elements (eigenvalues) from below with a threshold $\epsilon_q > 0$. Formally, if $s^k \in \mathbb{R}^n$ denotes these extracted diagonal entries, we set

$$B^k = \mathrm{D}(\max\{q + s^k, \epsilon_q\}), \quad \epsilon_q = 10^{-12}.$$

This guarantees $B^k \succ 0$. The clipping step biases the quadratic model, as directions with weak curvature are artificially regularized; however, this bias is controlled and helps stabilize the subproblems. We adopt this approach over quasi-Newton updates such as BFGS [15] for two main reasons:

- **Simplicity:** The diagonal extraction and clipping rule is extremely lightweight. It avoids storing or updating dense Hessian approximations, lowering both implementation complexity and memory footprint.
- **Parallelism:** A diagonal structure decouples coordinates and maps naturally to massively parallel hardware. This makes it highly efficient on GPUs, where dense matrix updates can become a bottleneck.

The tradeoff is clear: while BFGS can capture cross-term curvature, our diagonal approximation discards it entirely. For large-scale settings, however, this sacrifice is justified by the substantial gains in robustness and scalability.

Hessian Diagonal Computation To form B^k, we require only the diagonal entries of the curvature term

$$(y^k)^\top J_{h,xx}(x^k, \phi) \ \in \ \mathbb{R}^{n \times n},$$

where $y^k = y_+^k - y_-^k$. The (i,i)-th entry of this matrix is

$$s_i^k = e_i^\top \left((y^k)^\top J_{h,xx}(x^k, \phi) \right) e_i,$$

where e_i denotes the i-th standard basis vector in $\mathbb{R}^n$. Computing all second derivatives explicitly would require materializing the full Hessian tensor $J_{h,xx}$. Instead, we use the *forward-over-reverse* trick:
1. Compute the gradient of the scalar function

$$g(x) = \frac{d}{dx} \, (y^k)^\top h(x, \phi) \in \mathbb{R}^n$$

via reverse-mode AD, "hiding" the inner p dimension which is usually larger than n in the process.
2. For each coordinate direction e_i, compute the directional derivative

$$s_i^k = e_i^\top (J_{g,x} \, e_i)$$

using forward-mode AD and select the i-th element.
This requires n parallel forward sweeps in total. Each sweep computes the full vector $J_{g,x} \, e_i \in \mathbb{R}^n$, but we only retain its i-th component. Thus, the arithmetic cost per sweep is comparable to computing a full Hessian column, but the storage cost is reduced from $O(n^2)$ down to $O(n)$, since we never materialize or store the full matrix. The key advantage is scalability: the n sweeps are independent and can be run in parallel, making the procedure highly GPU-friendly. In practice, this means we trade away cross-term information, but we gain a lightweight and memory-efficient diagonal curvature estimate suitable for large-scale problems.

4.1.2 Quadratic programming subproblem

Using the positive definite Hessian approximation $B^k \succ 0$ from Subsection 4.1.1, we can form the QP subproblems following the standard SQP approach outlined in Section 3.4. At iteration k, this yields the quadratic program in the step variable $d_x \in \mathbb{R}^n$:

$$\min_{d_x \in \mathbb{R}^n} \quad \tfrac{1}{2} d_x^\top \mathrm{D}(q^k) d_x + (c^k)^\top d_x \tag{4.5a}$$

$$\text{s.t.} \quad l^k \leq A^k d_x \leq u^k, \tag{4.5b}$$

where the quantities are defined as

$$q^k = \max\{q + s^k,\, \epsilon_q\}, \tag{4.5c}$$

$$c^k = \mathrm{D}(\max\{q,\, \epsilon_q\}) x^k + c, \tag{4.5d}$$

$$l^k = l - h(x^k, \phi), \tag{4.5e}$$

$$u^k = u - h(x^k, \phi), \tag{4.5f}$$

$$A^k = J_{h,x}(x^k, \phi). \tag{4.5g}$$

By solving (4.5) using OSQP (Section 3.5) we obtain the primal dual optimal solution $[d_x^k, y^k]$ of this subproblem. The proposed update step to the current iterate $p^k = [x^k, y_+^k, y_-^k]$ then reads:

$$d_x^k, \tag{4.6a}$$

$$d_{y_+}^k = \max(y^k, 0) - y_+^k, \tag{4.6b}$$

$$d_{y_-}^k = \max(-y^k, 0) - y_-^k. \tag{4.6c}$$

4.1.3 Line search

In our concrete problem, the ℓ_1-merit function (3.8) is:

$$\phi(x; \rho) = \frac{1}{2} x^\top \mathrm{D}(\max\{q,\, \epsilon_q\}) x + c^\top x + \rho \| \max\{0, h(x, \phi) - u, l - h(x, \phi)\} \|_1.$$

Since the maximum function is not continuously differentiable, we cannot compute the exact gradient of the merit function at the constraint boundary. Instead, we use a directional derivative approximation consistent with the approach described in [3]. The directional derivative of the merit function along the search direction d_x is approximated by:

$$J_{\phi,x}(x; \rho) d_x = (D(\max\{q,\, \epsilon_q\}) x + c^\top x)^\top d_x - \rho \| \max\{0, h(x, \phi) - u, l - h(x, \phi)\} \|_1.$$

In our implementation, we use a backtracking line search to find an appropriate step size. We initialize the step size with $\beta = 1$ and a sufficient decrease parameter of $\eta = 10^{-4}$. We then test the sufficient decrease condition:

$$\phi(x^k + \beta^k d_x^k; \rho) \;\leq\; \phi(x^k; \rho) + \eta \beta^k J_{\phi,x}(x; \rho) d_x,$$

If the condition is satisfied, the step is accepted. Otherwise, we shrink the step size $\beta \leftarrow \beta * 75\%$ and repeat the process until the condition is met. This iterative strategy ensures a rapid and reliable convergence toward a step size that provides a sufficient decrease in the merit function. With step size β^k determined by a line search in direction d_x, the complete iterate is updated using this stepsize:

$$p^{k+1} = p^k + \beta^k d^k.$$

4.1.4 Convergence Criteria

The SQP algorithm seeks an iterate p that satisfies the first-order optimality conditions (4.3). The algorithm terminates when the current iterate p^k satisfies these KKT conditions within prescribed tolerances.

Our convergence assessment focuses on stationarity (*dual residual*) and primal feasibility (*primal residual*). Several considerations justify this approach. First, dual feasibility is satisfied by construction within every QP subproblem. Second, when primal feasibility and stationarity residuals approach zero, complementary slackness conditions typically emerge automatically. As the algorithm converges to the optimal solution where constraints become either clearly active or inactive, the complementary slackness condition tends to be satisfied implicitly without explicit monitoring. Third, explicit verification of complementary slackness involves evaluating products of potentially small quantities, introducing numerical sensitivity that may compromise convergence detection.

The termination criteria are formulated as:

$$\|r^k_{\text{prim}}\|_\infty \leq \epsilon_{\text{prim}} \quad \text{where } r^k_{\text{prim}} = \max(h(x^k,\phi) - u,\ l - h(x^k,\phi),\ 0), \tag{4.7a}$$

$$\|r^k_{\text{dual}}\|_\infty \leq \epsilon_{\text{dual}} \quad \text{where } r^k_{\text{dual}} = \mathrm{D}(q)x + c + J_{h,x}(x,\phi)^\top (y_+ - y_-), \tag{4.7b}$$

where ϵ_{prim} and ϵ_{dual} denote the prescribed tolerance parameters for primal feasibility and stationarity, respectively. The primal residual (4.7a) measures constraint violation, while the dual residual (4.7b) quantifies the violation of the stationarity condition derived from the Lagrangian optimality requirements.

4.2 Backward Pass

In the previous sections, we discussed how to solve the KKT conditions (4.3), obtaining the iterate $p^* = \begin{bmatrix} x^* & y_+^* & y_-^* \end{bmatrix}$ given the constraint parameter ϕ. This provides a viable strategy to solve the *forward pass* and obtain an optimal value $p^*(\phi)$.

In this section, we apply the IFT theory from Section 3.3 to compute a VJP for the *backward pass*, i.e., how sensitivities of the output (the optimal solution $p^*(\phi)$) propagate back to sensitivities with respect to the input (the constraint parameter ϕ). The implicit function theorem used in our backward pass *requires* that the forward pass has converged to a valid solution. If this is not the case, for example, because the forward pass has exceeded the maximum number of SQP iterations, we suppress gradients in the backward pass ($w_\phi = 0$).

Otherwise, we proceed as follows:

$$K(\phi, p) = \begin{bmatrix} D(q)x + c + J_{h,x}(x,\phi)^\top (y_+ - y_-) \\ D(y_+)(h(x,\phi) - u) \\ D(y_-)(l - h(x,\phi)) \end{bmatrix} = 0, \tag{4.8a}$$

where $K(\phi, p)$ collects the *equality-type* components of the KKT system (4.3). This subsystem is always satisfied whenever the full KKT conditions hold. Next, we form the Jacobian:

$$J_{K,p}(\phi, p^*) = \begin{bmatrix} D(q) + J_{h,xx}(x^k,\phi)^\top y^k & J_{h,x}(x^k,\phi)^\top & -J_{h,x}(x^k,\phi)^\top \\ D(y_+^k) J_{h,x}(x^k,\phi) & D(h(x^k,\phi) - u) & 0 \\ -D(y_-^k) J_{h,x}(x^k,\phi) & 0 & D(l - h(x^k,\phi)) \end{bmatrix}. \tag{4.8b}$$

Since we do not propagate the dual multipliers y out of the optimization layer, they have zero incoming downstream gradients. The full incoming adjoint is therefore:

$$v = \begin{bmatrix} v_1 \\ 0 \\ 0 \end{bmatrix}.$$

Solve Linear System to obtain Auxiliary Vector u We proceed with (3.3) by applying the same Hessian approximation used in the forward pass $D(q)+J_{h,xx}(x^*,\phi)^\top y^* \approx \mathrm{D}(\max(q+s^*,\epsilon_q))$ and transposing (4.8b) to obtain the linear system:

$$\left[\begin{array}{c|cc} \mathrm{D}(\max(q+s^*,\epsilon_q)) & +J_{h,x}(x^*,\phi)^\top \mathrm{D}(y_+^*) & -J_{h,x}(x^*,\phi)^\top \mathrm{D}(y_-^*) \\ \hline +J_{h,x}(x^*,\phi) & \mathrm{D}(h(x^*,\phi)-u) & 0 \\ -J_{h,x}(x^*,\phi) & 0 & \mathrm{D}(l-h(x^*,\phi)) \end{array}\right] \left[\begin{array}{c} u_1 \\ \hline u_2 \\ u_3 \end{array}\right] = \left[\begin{array}{c} -v_1 \\ \hline 0 \\ 0 \end{array}\right] \tag{4.9}$$

We will now leverage the block structure to simplify the problem as much as possible, without knowing any concrete values.

$$\begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} \begin{bmatrix} u_1 \\ [u_2, u_3] \end{bmatrix} = \begin{bmatrix} -v_1 \\ 0 \end{bmatrix} \tag{4.10a}$$

$$B_5 = B_4 - \epsilon_{\text{prim}} I \tag{4.10b}$$

Here we subtracted the regularizer ϵ_{prim}, to make B_5 negative definite and always invertible. This is the case since $l \le h$ and $h \le u$ on convergence, but only up to the tolerance ϵ_{prim}.

$$[u_2, u_3] = -B_5^{-1} B_3 u_1 \tag{4.10c}$$

$$(B_1 - B_2 B_5^{-1} B_3) u_1 = -v_1 \tag{4.10d}$$

We will now substitute the block contents of (4.9) into (4.10d) to obtain:

$$\left[\mathrm{D}(\max(q+s^*,\epsilon_q)) + J_{h,x}^\top \mathrm{D}\left(\frac{y_+^*}{\epsilon_{\text{prim}} + u - h^*} + \frac{y_-^*}{\epsilon_{\text{prim}} + h^* - l}\right) J_{h,x}\right] u_1 = -v_1. \tag{4.10e}$$

It can be verified that the LHS of (4.10e) is always positive definite; we therefore employ conjugate gradient descent (CG). This is implemented without forming the full Jacobian, but instead only VJPs and JVPs. The remaining components of u are found by applying (4.10c):

$$u_2 = \mathrm{D}\left(\frac{1}{\epsilon_{\text{prim}} + u - h(x^*,\phi)}\right) J_{h,x}(x^*,\phi) u_1, \tag{4.10f}$$

$$u_3 = \mathrm{D}\left(\frac{1}{l - \epsilon_{\text{prim}} - h(x^*,\phi)}\right) J_{h,x}(x^*,\phi) u_1. \tag{4.10g}$$

Perform VJP to obtain Adjoint Vector w Having obtained the full u we can proceed with (3.4), the final VJP:

$$w_\phi^\top = u^\top J_{K,\phi}(\phi, p^*), \tag{4.11a}$$

where w_ϕ is the desired adjoint with respect to ϕ. Since $K(\phi,p)$ (4.8a) is known explicitly, this step is performed using the "classical" VJP (Section 3.2) machinery.

$$J_{K,\phi}(\phi,p^*) = \begin{bmatrix} J_{h,x\phi}(x^*,\phi)^\top (y_+^* - y_-^*) \\ +\mathrm{D}(y_+) J_{h,\phi}(x^*,\phi) \\ -\mathrm{D}(y_-) J_{h,\phi}(x^*,\phi) \end{bmatrix} \tag{4.11b}$$

This means the adjoint w_ϕ can be obtained as the sum of three VJPs. We begin with the first:

$$w_{\phi 1} = \left(u_1^\top J_{h,x\phi}(x^*,\phi)^\top (y_+^* - y_-^*)\right)^\top \tag{4.11c}$$

$$= (y_+^* - y_-^*)^\top J_{h,\phi x}(x^*,\phi)\, u_1 \tag{4.11d}$$

$$= \frac{\partial}{\partial \phi}\left(y^{*\top}\left(\left(\frac{\partial h(x,\phi)}{\partial x}\right)_{x^*} u_1\right)\right)_\phi, \tag{4.11e}$$

where the last equation is a *reverse-over-forward* HVP, preferable for the usual case where $m > n$.

$$w_{\phi 2}^\top = u_2^\top \mathrm{D}(+y_+) J_{h,\phi}(x^*, \phi) \tag{4.11f}$$

$$w_{\phi 3}^\top = u_3^\top \mathrm{D}(-y_-) J_{h,\phi}(x^*, \phi) \tag{4.11g}$$

Finally, all three components are added:

$$w_\phi = w_{\phi 1} + w_{\phi 2} + w_{\phi 3}. \tag{4.11h}$$

4.3 Algorithm Summary

This section summarizes the SQP layer algorithm.
In our implementation, primal and dual tolerances are set to a common value,

$$\epsilon_{\mathrm{SQP}} = \epsilon_{\mathrm{SQP,prim}} = \epsilon_{\mathrm{SQP,dual}}, \quad \epsilon_{\mathrm{OSQP}} = \epsilon_{\mathrm{OSQP,prim}} = \epsilon_{\mathrm{OSQP,dual}}.$$

The inner OSQP solver must run at a higher accuracy than the outer SQP loop to provide valid search directions. We therefore enforce

$$\epsilon_{\mathrm{OSQP}} = 0.1 \cdot \epsilon_{\mathrm{SQP}}.$$

Both the outer SQP routine and the inner OSQP routine are terminated either once their respective convergence criteria are met or once a maximum iteration budget is exceeded.
Finally, to reduce overhead from repeatedly computing residuals and updating the penalty parameter ρ, the update step (3.13) is applied for several *group iterations* in succession before these checks are performed.

Algorithm 1: SQP Optimization Layer — *forward pass*

Input: ϕ, $q \geq 0$, c, l, u
Output: x^*

$x \leftarrow x_0$;
$y \leftarrow 0$;
$i \leftarrow 0$;
$j \leftarrow 0$;
while *SQP not converged (Section 4.1.4) and* $i <$ *max SQP iterations* **do**
    $i \leftarrow i + 1$;
    $\hat{q} \leftarrow$ diagonal Hessian approximation (Section 4.1.1);
    $\hat{c} \leftarrow \mathrm{D}(q)x + c$;
    $\hat{l} \leftarrow l - h(x, \phi)$;
    $\hat{u} \leftarrow u - h(x, \phi)$;
    $\hat{A} \leftarrow J_{h,x}(x, \phi)$;
    $\hat{x} \leftarrow 0$, $\hat{y} \leftarrow 0$, $\hat{\rho} \leftarrow \rho$;
    while *OSQP not converged (Section 3.5) and* $j <$ *max OSQP iterations* **do**
        $j \leftarrow j +$ *group iterations*;
        for *group iterations* **do**
            update OSQP variables using CG and (3.13);
        end
        $\hat{\rho} \leftarrow$ update penalty parameter ρ using (3.16);
    end
    $j \leftarrow 0$;
    $d_x \leftarrow \hat{x}$;
    $d_y \leftarrow \hat{y} - y$;
    $\beta \leftarrow$ merit function line search (Section 4.1.3);
    $x \leftarrow x + \beta d_x$;
    $y \leftarrow y + \beta d_y$;
end

Algorithm 2: SQP Optimization Layer — *backward pass*

Input: $\bar{x}$
Data: q, c, l, u, ϕ, x^*, y^*
Output: $\bar{\phi}$

$y^*_+ \leftarrow \max(+y^*, 0)$;
$y^*_- \leftarrow \max(-y^*, 0)$;
$u_1 \leftarrow$ solve (4.10e) using CG;
$u_2 \leftarrow$ compute (4.10f) using a VJP;
$u_3 \leftarrow$ compute (4.10g) using a VJP;
$\bar{\phi} \leftarrow$ compute (4.11a) using a HVP and two VJPs;

Chapter 5

Rollout Projection

5.1 Control Problem

We consider a discrete-time dynamical system with known, potentially nonlinear dynamics, described by the transition function

$$s_{t+1} = f(s_t, a_t), \quad f : \mathbb{R}^{n_s} \times \mathbb{R}^{n_a} \to \mathbb{R}^{n_s},$$

where $s_t \in \mathbb{R}^{n_s}$ denotes the state at time t and $a_t \in \mathbb{R}^{n_a}$ the control input.
Given a finite horizon n_h, a *rollout* (s, a) consists of a sequence of actions $a \in \mathbb{R}^{n_h \times n_a}$ and the corresponding sequence of states $s \in \mathbb{R}^{n_h \times n_s}$ obtained by applying the transition function starting from an initial state $s_0 \in \mathbb{R}^{n_s}$.
The system is subject to (potentially nonconvex) state-action constraints

$$l \leq g(s, a) \leq u, \quad g : \mathbb{R}^{n_h \times n_s} \times \mathbb{R}^{n_h \times n_a} \to \mathbb{R}^{n_c},$$

with $l, u \in [\mathbb{R} \cup \{-\infty, +\infty\}]^{n_c}$. A rollout is further associated with a *trajectory cost* $\text{cost}(s, a)$, reflecting task-specific performance. The goal is to generate a feasible rollout of minimal cost starting from any admissible initial state s_0.

5.2 Learning Approach

Inspired by explicit model predictive control (EMPC, cf. Section 3.6), we propose a receding horizon learning-based architecture illustrated in Figure 5.1.
A neural network predicts a candidate action sequence $\hat{a}$ over the horizon n_h. This sequence is rolled out through the known dynamics, yielding a proposal $(\hat{s}, \hat{a})$. By construction, the dynamics are satisfied, but the proposal may violate state-action constraints. To ensure feasibility, the proposal is passed through the SQP layer, which projects it onto the feasible set and returns the closest admissible rollout (s, a). Only the first action of the corrected rollout is executed on the system, after which the horizon is shifted and the process repeats.

5.3 SQP Projection Layer

The projection step is formulated as the optimization problem:

$$\min_{s,a,\xi} \quad \|s - \hat{s}\|_2^2 + \|a - \hat{a}\|_2^2 + \lambda_1 \|\xi\|_2^2 \tag{5.1a}$$

$$\text{s.t.} \quad s_{t+1} = f(s_t, a_t), \quad t = 0, \ldots, n_h - 2, \tag{5.1b}$$

$$l \leq g(s, a) + \xi \leq u, \tag{5.1c}$$

$$s_0 = s_0^*, \tag{5.1d}$$

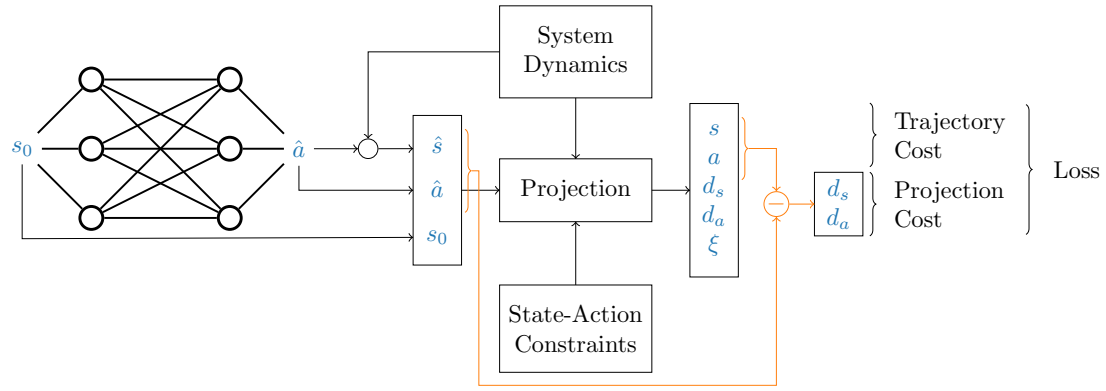


Figure 5.1: Receding Horizon Network Pipeline. The alteration shown in orange allows the projection cost to have meaningful gradients even if the projection hasn't met the convergence criteria.

where slack variables $\xi \in \mathbb{R}^{n_c}$ ensure the problem is always feasible, with their magnitude penalized by $\lambda_1 \in \mathbb{R}_{++}$.
We reformulate the problem using auxiliary deviation variables d_s and d_a:

$$\begin{aligned}
\min_{s,a,d_s,d_a,\xi} \quad & \|d_s\|_2^2 + \|d_a\|_2^2 + \lambda_1 \|\xi\|_2^2 && \text{(5.2a)}\\
\text{subject to} \quad & 0 \le s - \hat{s} - d_s \le 0, && \text{(5.2b)}\\
& 0 \le a - \hat{a} - d_a \le 0, && \text{(5.2c)}\\
& 0 \le s_{t+1} - f(s_t, a_t) \le 0 \quad \forall t \in [0, \text{horizon} - 1], && \text{(5.2d)}\\
& l \le g(s, a) + \xi \le u, && \text{(5.2e)}\\
& 0 \le s_0 - s_0^* \le 0. && \text{(5.2f)}
\end{aligned}$$

This separation of deviations, slack variables, and constraints is well-suited for SQP, as the projection input $\phi = (\hat{s}, \hat{a}, s_0^*)$ only appears in the parametric constraints $h(x, \phi)$.

5.4 Loss Function

The natural learning objective is the trajectory cost $\text{cost}(s, a)$. However, since the projection step is nonconvex, the convergence of SQP strongly benefits from proposals close to a local minimizer. As a matter of fact, it can be shown that SQP converges quadratically if the initial point is sufficiently close to a local minimizer [3]. To encourage the network to generate such proposals, we add an auxiliary *projection cost* weighted by $\lambda_2 \in \mathbb{R}_+$:

$$l(s, a, d_s, d_a) = \text{cost}(s, a) + \lambda_2 \big(\|d_s\|_2^2 + \|d_a\|_2^2 \big). \tag{5.3}$$

If the network learns to align its proposals with the projection, the deviations vanish and the auxiliary term becomes negligible.
In practice, SQP iterations may terminate before full convergence, particularly for long horizons or poor initializations. Although gradients through the projection (via the IFT) are only valid once the forward problem has converged, partially converged rollouts are still expected to be closer to feasibility than proposals, since SQP operates under the iterative refinement assumption. To exploit this, we substitute $d_s = s - \hat{s}$ and $d_a = a - \hat{a}$ directly, yielding the practical loss:

$$l(s, a, \hat{s}, \hat{a}) = \text{cost}(s, a) + \lambda_2 \big(\|s - \hat{s}\|_2^2 + \|a - \hat{a}\|_2^2 \big). \tag{5.4}$$

This adaptation allows alignment gradients to "flow around the projection".

5.5 Training Procedure

The network is trained by uniformly sampling initial states s_0 from a bounding set that contains, but is not limited to, the states from which feasible rollouts exist. This ensures exposure not only to feasible initializations but also to states that will lead to constraint violations. Since the projection layer incorporates slack variables, every sampled initialization yields a well-defined solution. When s_0 lies outside the strictly feasible region, the slack provides a recovery mechanism, allowing the network to learn corrective strategies for handling infeasible starting points.
Training then proceeds by minimizing the loss (5.4) with gradient descent. This simultaneously reduces the trajectory cost and encourages the network's proposals to align with feasible rollouts, thereby improving the quality of the SQP initialization and accelerating convergence during deployment.

Chapter 6

Results

6.1 Linear System - Convex Constraints

We begin with the simplest case of rollout projection to illustrate the core principles of our method. Consider the *linear* double-integrator system:

$$\begin{bmatrix} s_1^{k+1} \\ s_2^{k+1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} s_1^k \\ s_2^k \end{bmatrix} + \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} a_1^k, \tag{6.1}$$

subject to the box constraints

$$\begin{aligned} 1 \leq s_1 \leq 10, \\ -10 \leq s_2 \leq 10, \\ -3 \leq a_1 \leq 3. \end{aligned} \tag{6.2}$$

To obtain a raw rollout, we apply the controller $a^k = \begin{bmatrix} -0.2 & -\frac{2}{3} \end{bmatrix} s^k$, starting from the initial state $x^0 = \begin{bmatrix} 3 & 5 \end{bmatrix}^\top$. This trajectory is then passed through our SQP-based projection layer. Because both dynamics and constraints are linear, the projection converges in a single SQP iteration, yielding the corrected rollout:

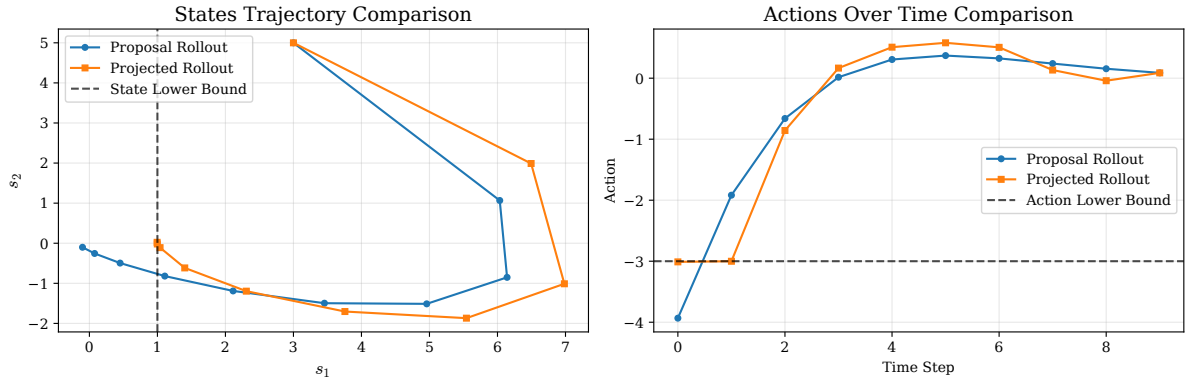


Figure 6.1: Comparison of raw and projected rollouts for the linear system with binding state and action constraints.

As shown in Figure 6.1, the projected trajectory satisfies all state and input constraints while minimizing deviation from the original rollout at each timestep.
We next demonstrate a VJP calculation. As recalled from Section 3.2, the VJP encodes how an infinitesimal change in the output (in direction v) propagates back to the input. Figure 6.2

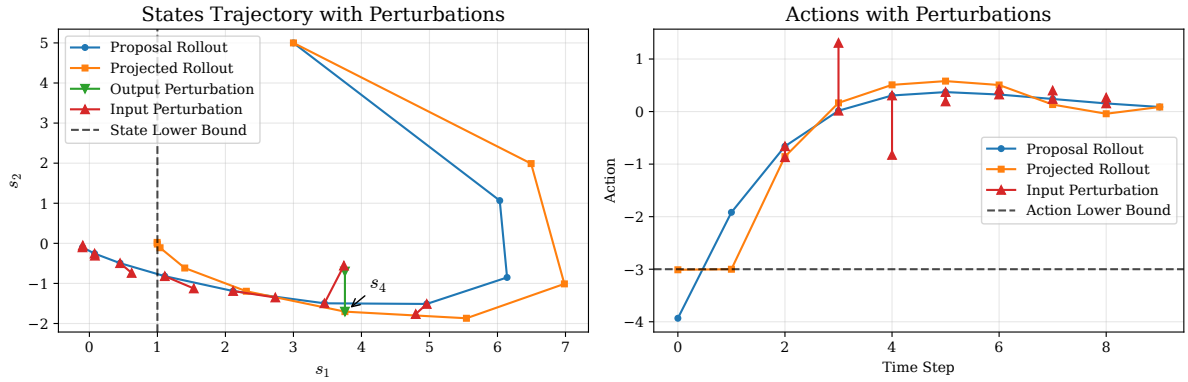


Figure 6.2: The VJP relates output perturbations, here an indicator at s_4, to input perturbations. We see that binding constraints predetermine the projection output and eliminate sensitivity with respect to parts of the input.

illustrates the effect with the perturbation v being an indicator at s^4. For clarity, the perturbation magnitude is deliberately exaggerated. Additionally, the resulting input perturbations are scaled up by a factor of 5 to improve visibility. Two observations follow: (i) the strongest state correlation arises, as expected, with $\hat{s}^4$; (ii) because the action saturates at $t = 0$ and $t = 1$, and the initial state is fixed, there is no sensitivity with respect to $\hat{s}^0, \hat{s}^1, \hat{s}^2$ or $\hat{a}^0, \hat{a}^1$, as these variables cannot influence the projected rollout's evolution under the given constraints.

6.2 Linear System - Nonconvex Constraints

We now consider the same linear system (6.1), but subject to a nonconvex state constraint. Specifically, we impose a circular exclusion zone centered at $s_c = \begin{bmatrix} 6.5 & 0 \end{bmatrix}$ with radius $r_c = 1.5$. The resulting projection is shown in Figure 6.3.

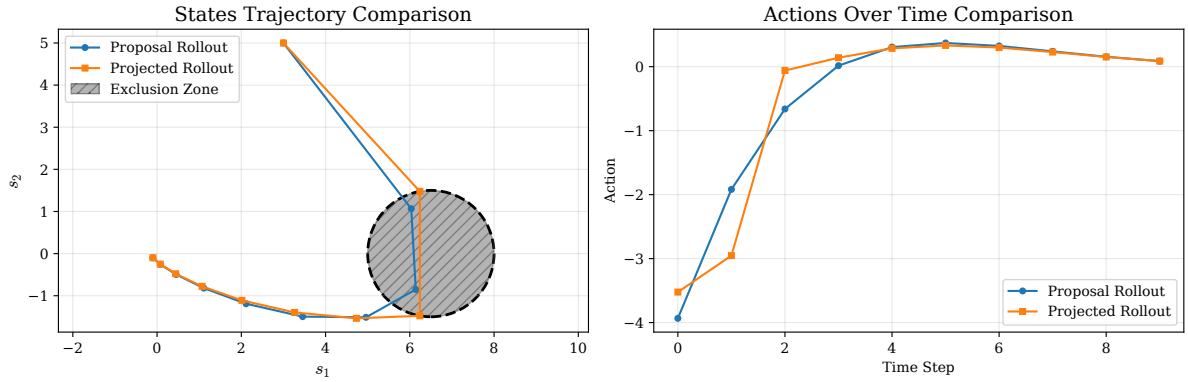


Figure 6.3: Projection with nonconvex constraint: all discrete states s_i satisfy the exclusion zone.

Forward pass verification Since the constraints $h(x, \phi)$ are nonlinear, multiple SQP iterations are required to reach convergence. Figure 6.4 illustrates the residual evolution with tolerance $\epsilon_{\text{SQP}} = 10^{-5}$. Two observations follow: (i) the accuracy of SQP is ultimately limited by the inner OSQP tolerance ($\epsilon_{\text{OSQP}} = 10^{-6}$), and (ii) primal residuals—measuring constraint

satisfaction—decay much faster than dual residuals, which correspond to the stabilization of Lagrange multipliers ("constraint pricing").

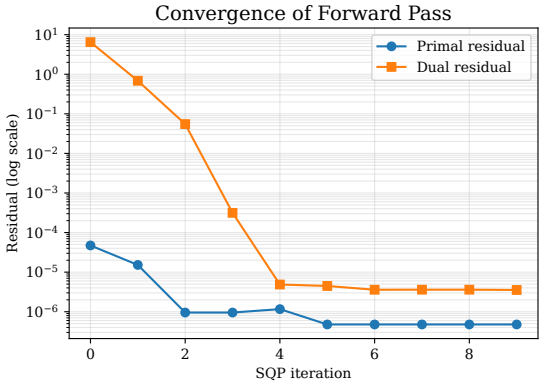


Figure 6.4: Forward pass verification: residual evolution with $\epsilon_{\text{SQP}} = 10^{-5}$ and $\epsilon_{\text{OSQP}} = 10^{-6}$.

Backward pass verification To verify the correctness of our custom VJP implementation, we follow the test strategy used in JAX [10]. The idea is to check whether the identity

$$\langle v, \text{JVP}(u) \rangle = \langle \text{VJP}(v), u \rangle$$

holds for randomly chosen test vectors u and v. Both sides evaluate to the same bilinear form $v^\top J u$, where J_k denotes the Jacobian of k. In this setting, the VJP corresponds to our implicit backward pass, while the JVP is approximated using a finite-difference directional derivative:

$$\text{JVP}(u) \approx \frac{f(x + \varepsilon u) - f(x - \varepsilon u)}{2\varepsilon}, \quad \varepsilon = 3 \cdot \epsilon_{\text{SQP}}.$$

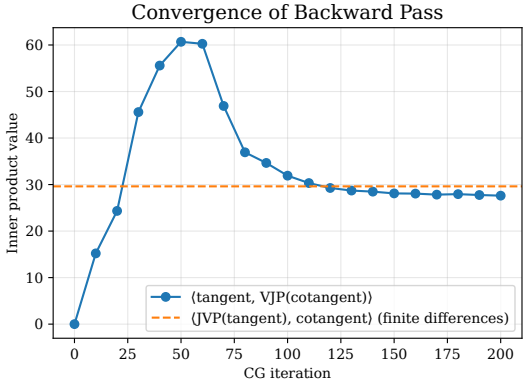


Figure 6.5: Backward pass verification: comparison of VJP against finite differences for nonconvex system. The x-axis shows the number of CG iterations used to solve (4.10e). Convergence is achieved after about 100 iterations, with a slight underestimation due to the regularization that makes constraints "appear less active".

6.3 Nonlinear System with Nonconvex Constraints

Next, we focus on a nonlinear system. Specifically, the kinematic bicycle model [12, p. 100]:

$$\dot{x} = v\cos(\theta), \tag{6.3a}$$
$$\dot{y} = v\sin(\theta), \tag{6.3b}$$
$$\dot{\theta} = \tfrac{v}{L}\tan(\gamma), \tag{6.3c}$$
$$\dot{v} = \dot{v}, \tag{6.3d}$$

where x and y denote the vehicle position in the world frame, θ is the heading angle relative to the x-axis, L is the wheelbase (distance between front and rear axles), v is the forward velocity, and γ is the steering angle. The corresponding turning radius is $R = L/\tan(\gamma)$.
We separate these variables into the state vector $s = \begin{bmatrix} x & y & \theta & v \end{bmatrix}$ and control inputs $a = \begin{bmatrix} \dot{v} & \gamma \end{bmatrix}$.

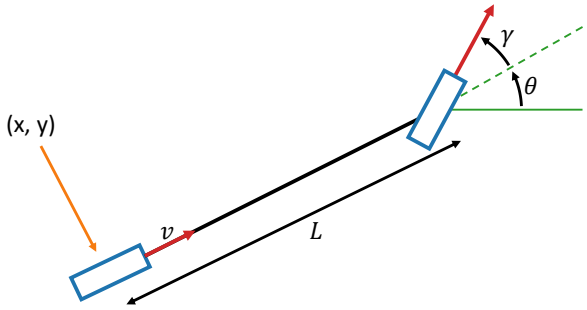


Figure 6.6: Illustration of kinematic bike model.

To apply our projection method, we discretize the continuous-time dynamics by employing a fourth-order Runge-Kutta [5] (RK4) scheme with step size Δt, constant control input during each interval, and ODE $f(s,a) = \begin{bmatrix} v\cos\theta & v\sin\theta & \frac{v}{L}\tan\gamma & \dot{v} \end{bmatrix}^\top$:

$$k_1 = f(s_k,\, a_k), \tag{6.4a}$$
$$k_2 = f\left(s_k + \tfrac{\Delta t}{2} k_1,\, a_k\right), \tag{6.4b}$$
$$k_3 = f\left(s_k + \tfrac{\Delta t}{2} k_2,\, a_k\right), \tag{6.4c}$$
$$k_4 = f\left(s_k + \Delta t\, k_3,\, a_k\right), \tag{6.4d}$$
$$s_{k+1} = s_k + \tfrac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right), \tag{6.4e}$$

For all following experiments we use $L = 1\,\text{m}$, steering limits $|\gamma| < 0.6\,\text{rad} \approx 35°$, and a fixed timestep of $\Delta t = 0.4\,\text{s}$. A raw rollout $(\hat{s}, \hat{a})$ is generated from the initial state $s_0 = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$ by applying constant inputs $a_i = \begin{bmatrix} 0 & 0 \end{bmatrix}$ for all $i \in [0, 49]$. This setup corresponds to the bicycle moving at a constant velocity of $1\,\text{m/s}$ along the positive x-axis, with no steering or acceleration inputs, over a 50-step horizon. In addition to the steering limits, we impose three nonconvex exclusion zones in the state space, as shown in Figure 6.7.
Figure 6.8 provides a more detailed view of the projection. It highlights how the correction arises from an interplay among all state and action variables, whose deviations are *jointly* minimized. This explains, for example, why the xy-trajectory does not "trace along" the exclusion zone boundaries: the projection simultaneously prioritizes keeping the heading angle close to the reference $\theta = 0$, rather than optimizing position in isolation.

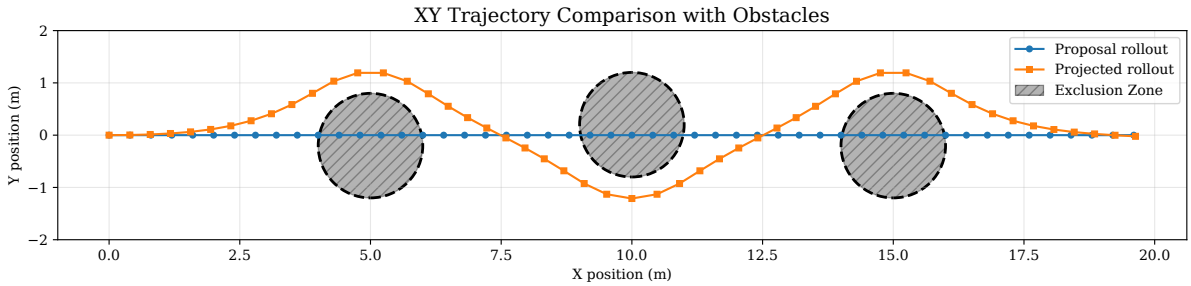


Figure 6.7: Comparison of raw and projected coordinate trajectories for the bike system.

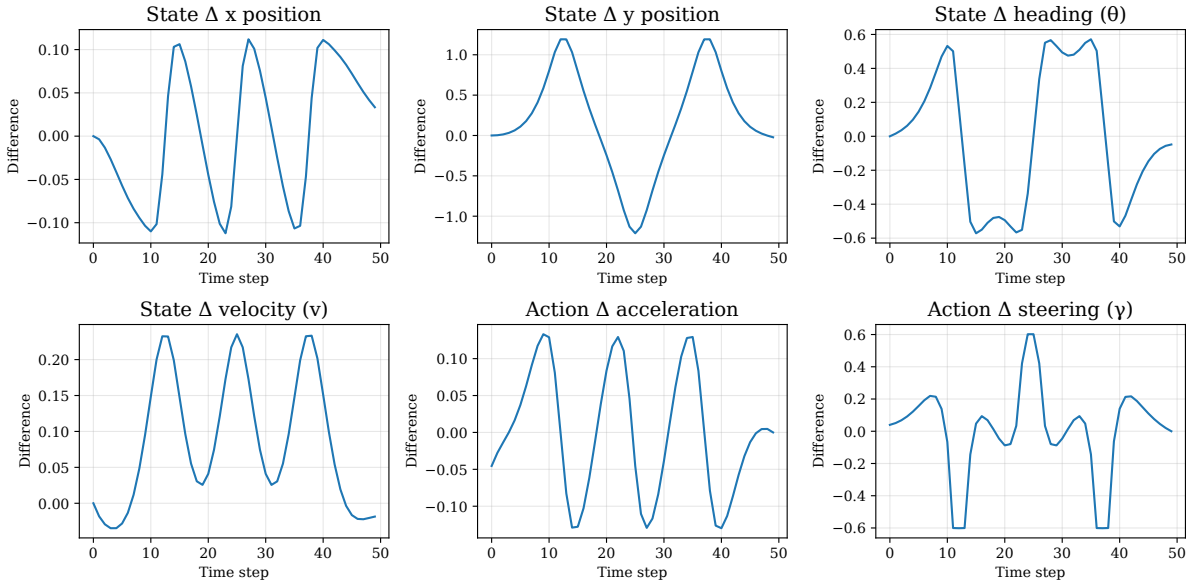


Figure 6.8: Difference between the bike rollout proposal and the projected trajectory in states d_s and actions d_a.

Forward Pass Verification. Figure 6.9 shows the residual evolution for $\epsilon_{\text{SQP}} = 10^{-2}$. Consistent with the observations in Section 6.2, primal residuals converge substantially faster than dual residuals. Both residuals plateau once the inner OSQP tolerance $\epsilon_{\text{OSQP}} = 10^{-3}$ is reached.

Backward Pass Verification. We repeat the VJP consistency test from Section 6.2. As shown in Figure 6.10, the larger tolerance ϵ_{SQP} used for this more complex problem introduces stronger regularization, which results in a systematic underestimation of the VJP.

6.4 Nonconvex Clover Projection

To study the behavior of the backward pass across a larger set of instances, we introduce the *clover set projection*, which enables visualization and analysis of sensitivities over the entire

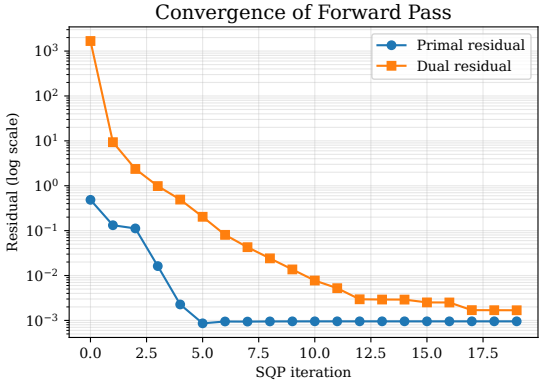


Figure 6.9: Forward pass verification: nonconvex constraints, nonlinear bike system rollout projection with $\epsilon_{\text{SQP}} = 10^{-2}$ and $\epsilon_{\text{OSQP}} = 10^{-3}$.

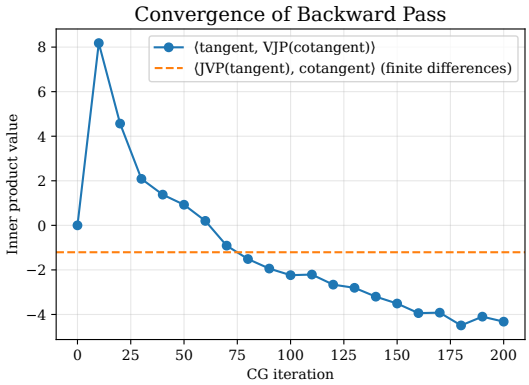


Figure 6.10: Backward pass verification: comparison of VJP against finite differences for the bike system. The x-axis indicates the number of CG iterations used to solve (4.10e).

two-dimensional state space. The clover constraint set is defined as

$$(x^2 + y^2)^2 - x^3 + 3xy^2 \;\leq\; 10^{-3}.$$

We now repeat the backward pass gradient analysis from Section 6.2, this time on 2000 projection instances within the clover setting. The results are presented in Figure 6.12. Inspection of the histogram comparing forward $\langle \text{JVP}(v), u \rangle$ and reverse $\langle v, \text{VJP}(u) \rangle$ evaluations reveals a small but distinct cluster of outliers. In these cases, the finite-difference JVP yields extreme values. Figure 6.13 highlights the corresponding points in the state space: they occur where the two finite-difference evaluations are projected onto different clover leaves, resulting in large discrepancies. Since the derivative truly is discontinuous at such boundaries, we discard these samples from further analysis.
Focusing on the filtered data, we observe good agreement between the VJP and the finite-difference JVP.
This experiment confirms that our backward pass remains consistent with finite-difference estimates, except at points of discontinuity that are intrinsic to the nonconvex projection.

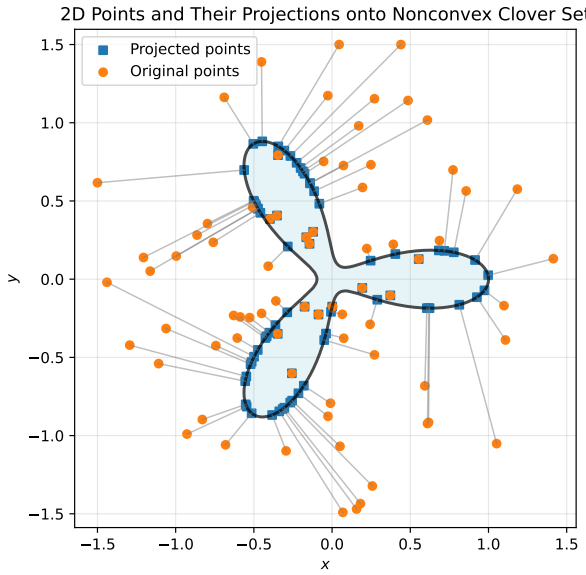


Figure 6.11: Nonconvex projection onto the clover set. Neighboring points may be projected into different leaves, highlighting the inherent nonconvexity of the problem.

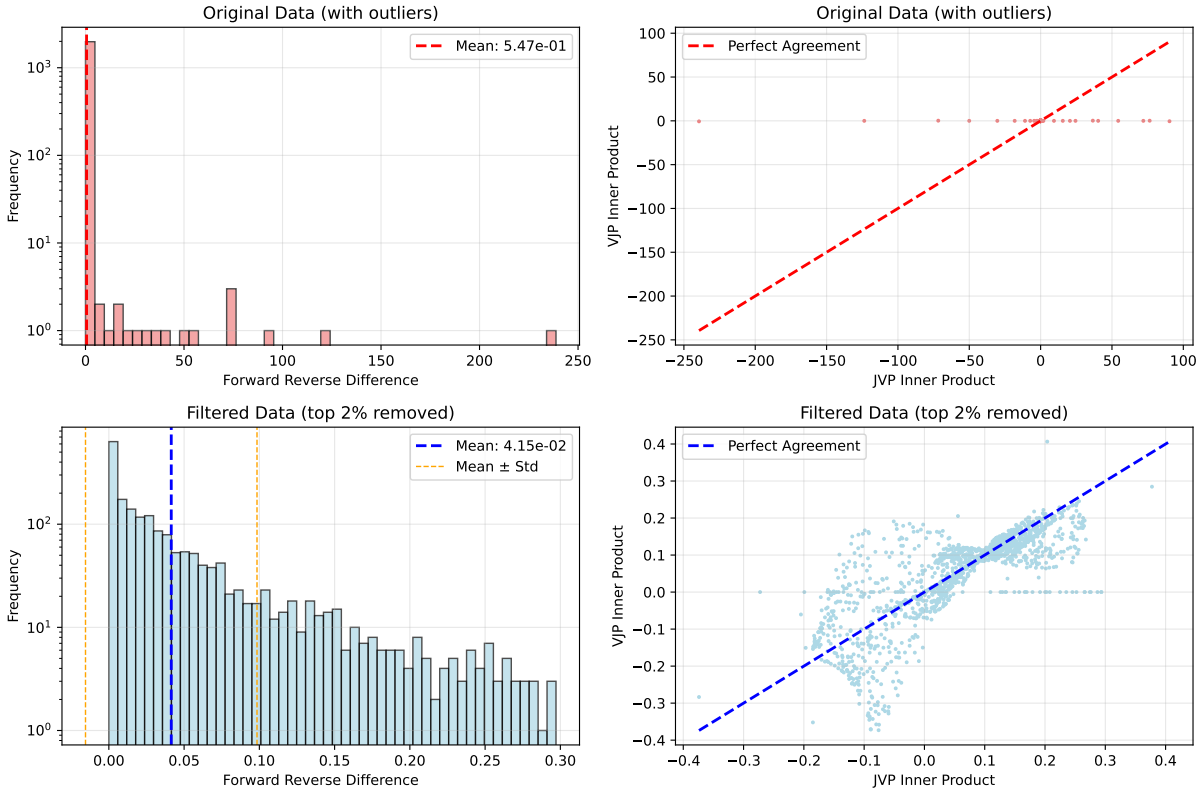


Figure 6.12: Histogram of forward $\langle \mathrm{JVP}(v), u \rangle$ and reverse $\langle v, \mathrm{VJP}(u) \rangle$ differences. After filtering out discontinuous regions, the VJP closely matches the finite-difference JVP. The line of zero valued VJP are problems that don't meet convergence citeria.

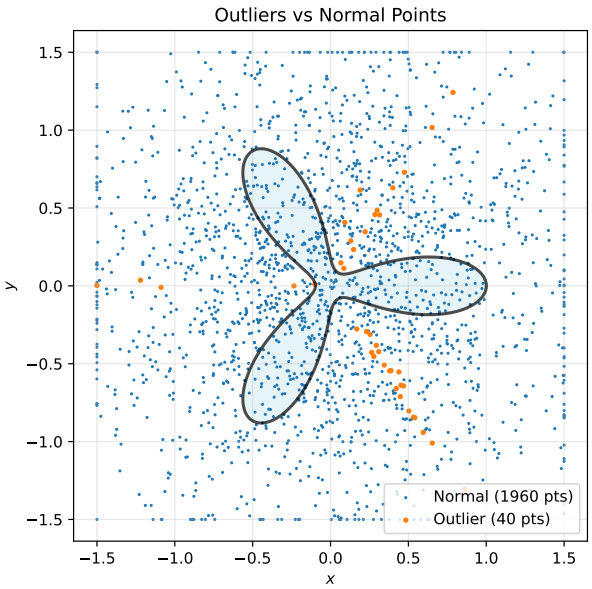


Figure 6.13: Finite-difference JVP failure cases. At discontinuities, the two evaluations are projected onto different clover leaves.

Chapter 7

Conclusion

This thesis introduces a novel SQP-based projection layer that enforces hard nonconvex constraints in neural networks while maintaining end-to-end differentiability. The approach formulates constraint satisfaction as parametric optimization problems solved via Sequential Quadratic Programming, with gradients computed using the Implicit Function Theorem to avoid the instability of unrolled optimization.

Contributions We developed a rigorous framework that leverages second-order curvature information through diagonal Hessian approximation, enabling faster convergence than first-order methods. The merit function-based line search improves convergence characteristics even from poor initializations. Experimental validation across linear and nonlinear systems with convex and nonconvex constraints demonstrates the method's effectiveness in rollout projection for control applications, where neural networks propose trajectories that are projected onto feasible sets defined by system dynamics and safety constraints.

Limitations The method faces several key limitations. Convergence is only guaranteed to local optima, with solution quality depending on initialization from neural network proposals. The diagonal Hessian approximation discards cross-term curvature information that could improve convergence rates. Computational overhead arises from nested optimization requiring sequential quadratic program solutions, while the backward pass demands additional memory for intermediate variables and linear system solutions. Practical deployment requires careful tuning of multiple hyperparameters, including tolerance levels, penalty parameters, and line search settings.

Algorithmic Improvements Future work should explore adaptive Hessian approximations that balance computational efficiency with curvature accuracy, potentially using limited-memory BFGS or structured approximations capturing key cross-terms while maintaining sparsity. Intelligent warm-starting strategies leveraging problem structure or historical solutions could reduce SQP iterations, particularly valuable in receding horizon applications where consecutive problems exhibit similarity. Multi-level optimization approaches that utilize coarse-grained projections to initialize fine-grained refinements could enhance scalability for higher-dimensional problems.

Application Domains The framework could extend to robust control through uncertainty handling in system dynamics and constraints via robust optimization techniques. Multi-agent coordination problems with inter-agent constraints represent another promising direction for safe distributed learning. Physics-informed neural networks could ensure

exact satisfaction of physical laws rather than approximate adherence through penalty methods.

The SQP layer framework provides a principled foundation for deploying learning-based methods in safety-critical applications where constraint violations are unacceptable. While computational overhead and local optimality limitations remain, the guarantee of constraint satisfaction often justifies these costs in domains requiring mathematical safety assurance.

Bibliography

[1] Akshay Agrawal et al. *Differentiable Convex Optimization Layers*. arXiv:1910.12430 [cs]. Oct. 2019.

[2] Brandon Amos and J. Zico Kolter. *OptNet: Differentiable Optimization as a Layer in Neural Networks*. arXiv:1703.00443 [cs]. Dec. 2021.

[3] Neculai Andrei. "Sequential Quadratic Programming". en. In: *Modern Numerical Nonlinear Optimization*. Ed. by Neculai Andrei. Cham: Springer International Publishing, 2022, pp. 521–567.

[4] Larry Armijo. "Minimization of functions having Lipschitz continuous first partial derivatives." In: *Pacific Journal of Mathematics* 16.1 (1966). Publisher: Pacific Journal of Mathematics, A Non-profit Corporation, pp. 1 –3.

[5] Gregory L. Baker and Jerry P. Gollub. "Numerical integration – Runge–Kutta method". In: *Chaotic Dynamics: An Introduction*. Cambridge University Press, 1996, pp. 193–195.

[6] Alberto Bemporad et al. "The explicit linear quadratic regulator for constrained systems". In: *Automatica* 38.1 (2002), pp. 3–20.

[7] Francesco Borrelli et al. "On the complexity of explicit MPC laws". In: *2009 European Control Conference (ECC)*. 2009, pp. 2408–2413.

[8] Stephen Boyd. "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers". en. In: *Foundations and Trendső in Machine Learning* 3.1 (2010), pp. 1–122.

[9] Stephen P. Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge, UK ; New York: Cambridge University Press, 2004.

[10] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018.

[11] Steven Chen et al. "Approximating Explicit Model Predictive Control Using Constrained Neural Networks". In: *2018 Annual American Control Conference (ACC)*. ISSN: 2378-5861. June 2018, pp. 1520–1527.

[12] Peter Corke. *Robotics, Vision and Control*. Vol. 118. Springer Tracts in Advanced Robotics. Cham: Springer International Publishing, 2017.

[13] Asen L. Dontchev and R. Tyrrell Rockafellar. *Implicit Functions and Solution Mappings: A View from Variational Analysis*. en. Springer Monographs in Mathematics. New York, NY: Springer New York, 2009.

[14] Priya L. Donti, David Rolnick, and J. Zico Kolter. *DC3: A learning method for optimization with hard constraints*. en. arXiv:2104.12225 [cs]. Apr. 2021.

[15] Roger Fletcher. *Practical methods of optimization*. eng. Online-Ausg. Chichester, West Sussex, U.K: Wiley, 2008.

[16] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation, Second Edition*. en. Second. Society for Industrial and Applied Mathematics, Jan. 2008.

[17] Panagiotis D. Grontas et al. *Pinet: Optimizing hard-constrained neural networks with orthogonal projection layers*. arXiv:2508.10480 [cs]. Aug. 2025.

[18] George Em Karniadakis et al. "Physics-informed machine learning". en. In: *Nature Reviews Physics* 3.6 (June 2021). Publisher: Nature Publishing Group, pp. 422–440.

[19] Giacomo Lastrucci and Artur M. Schweidtmann. *ENFORCE: Exact Nonlinear Constrained Learning with Adaptive-depth Neural Projection*. arXiv:2502.06774 [cs]. Feb. 2025.

[20] Youngjae Min, Anoopkumar Sonar, and Navid Azizan. *Hard-Constrained Neural Networks with Universal Approximation Guarantees*. en. arXiv:2410.10807 [cs]. Oct. 2024.

[21] Jorge Nocedal. "Line Search Methods". en. In: *Numerical Optimization*. Series Title: Springer Series in Operations Research and Financial Engineering. Springer New York, 2006, pp. 30–65.

[22] Michel Schubiger, Goran Banjac, and John Lygeros. "GPU acceleration of ADMM for large-scale quadratic programming". In: *Journal of Parallel and Distributed Computing* 144 (Oct. 2020), pp. 55–67.

[23] Bartolomeo Stellato et al. "OSQP: an operator splitting solver for quadratic programs". en. In: *Mathematical Programming Computation* 12.4 (Dec. 2020), pp. 637–672.

[24] Jesus Tordesillas, Jonathan P. How, and Marco Hutter. *RAYEN: Imposition of Hard Convex Constraints on Neural Networks*. en. arXiv:2307.08336 [cs]. July 2023.

[25] Kim P. Wabersich and Melanie N. Zeilinger. *A predictive safety filter for learning-based control of constrained nonlinear dynamical systems*. arXiv:1812.05506 [cs]. May 2021.

[26] Brendt Wohlberg. *ADMM Penalty Parameter Selection by Residual Balancing*. arXiv:1704.06209 [math]. Apr. 2017.